

Как перестать отлаживать асинхронный код и начать жить

Андрей Саломатин
FrontendConf, Москва
21.05.2015

Schlecht!Script

Функции имеют цвет

```
function f1*red(a, b) {...}  
    f1*red(a, b)
```

```
function f2*blue(c) {...}  
    f2*blue(c)
```

Schlecht!Script

Синие могут вызывать только другие синие функции

```
/* OK! */
```

```
function outer*blue() {  
    inner*blue()  
}
```

```
/* NOT OK! */
```

```
function outer*blue() {  
    inner*red()  
}
```

Schlecht!Script

Красные могут вызывать и красные и синие функции

```
/* OK! */
```

```
function outer*red() {  
    inner2*red()  
}
```

```
/* OK! */
```

```
function outer*red() {  
    inner2*blue()  
}
```

Schlecht!Script

Писать и вызывать **красные**
функции больно!

Красные функции нужно
называть на немецком!

`/* Интерпретатор не поймёт */`

`function authUser*red() {...}`

`function getName*red() {...}`

Schlecht!Script

Красные функции нужно
называть на немецком!

`/* RICHTIG! */`

`function benutzerAutorisierung!*rot() {...}`

`function nameErhalten!*rot() {...}`

Schlecht!Script

Как писать на Schlecht!Script?

JavaScript

JavaScript

extends

Schlecht!Script

Асинхронные функции
это боль

Асинхронные функции это боль

- По-другому работают if/else, for, return
- Нет try/catch
- Ломают абстракцию

JavaScript

if/for синхронно

```
// выполнять последовательно  
for (var i = 0; i < 10; i++) {  
    if (shouldProcess(i)) {  
        results.push(process(i));  
    }  
}
```

JavaScript

if/for асинхронно

```
function maybeProcess(i) {  
  if (i >= 10) { return; }  
  
  shouldProcess(function(should) {  
    if (should) {  
      process(i, function(result) {  
        results.push(result);  
  
        maybeProcess(i++);  
      });  
    }  
  
    maybeProcess(i++);  
  });  
}  
  
maybeProcess(0);
```

JavaScript

Асинхронные функции это боль

- Не работают if/then, for и т.д.
- Нет try/catch
- Ломают абстракцию

JavaScript



Асинхронность

в JavaScript

Андрей Саломатин
@filipovskii

Productive Mobile
MoscowJS
RadioJS



Асинхронность

Асинхронность: два сценария

Множество событий

Единиичная операция



Контроль

Исключения

Единый интерфейс



Контроль Исключения Единый интерфейс



YES WE CAN

ES6

- Работа с множеством событий
EventEmitter
Stream
- Работа с асинхронными операциями
Continuation Passing Style
Promises
Coroutines

ES7

- Работа с множеством событий
Async Generators
- Работа с асинхронными операциями
Async/Await

ES6

Работа с множеством асинхронных событий

Работа с множеством асинхронных событий

EventEmitter

Stream

Объект — источник событий

EventEmitter

Примеры

Browser: XMLHttpRequest

Node: http.Server

EventEmitter

API

```
emitter.addListener(eventName, cb);  
emitter.removeListener(eventName, cb);
```

EventEmitter

События XMLHttpRequest

progress (n)

load (1)

abort (1)

error (1)

EventEmitter

Реализации

Браузер + Node:

EventEmitter3, Tiny Emitter

Node:

Node EventEmitter

EventEmitter

Работа с множеством асинхронных событий

EventEmitter

Stream

Работа с множеством асинхронных событий

EventEmitter

Stream

Поток данных

Stream

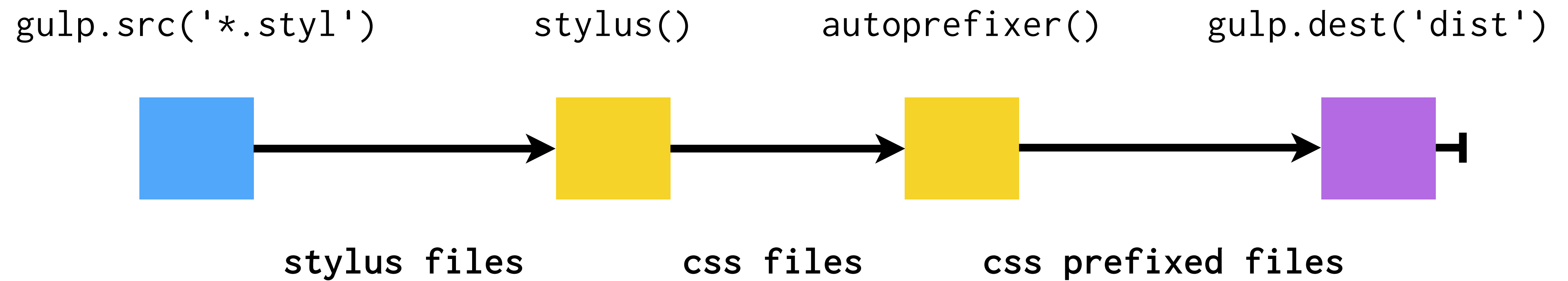
Примеры

Node: `fs.createReadStream(path)`

Node: `gulp.src(pattern)`

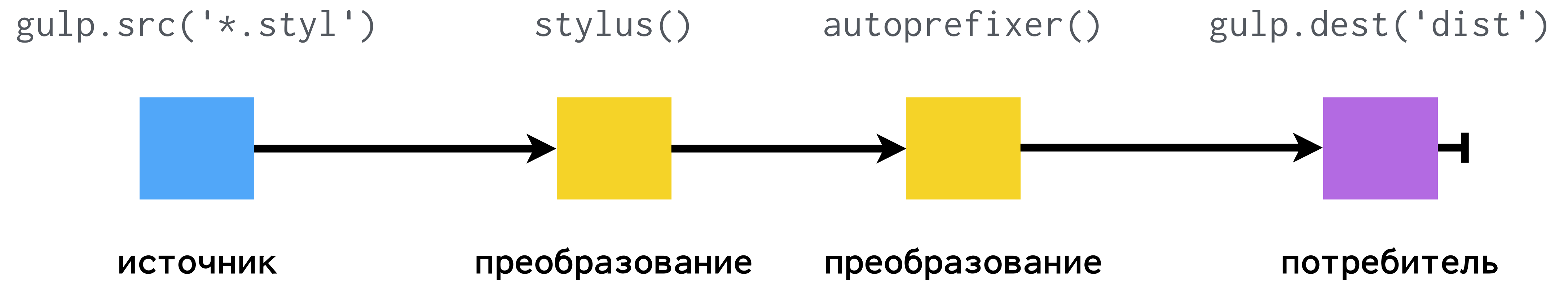
Stream

Типы потоков



Stream

Типы потоков



Stream



Реализации

Изоморфные:

RxJS, Kefir, Bacon.js

Node:

Node Streams

Stream

Работа с МНОЖЕСТВОМ

EventEmitter

Stream

EventEmitter, Stream

Контроль

Исключения

Единый интерфейс

Работа с асинхронными операциями

Работа с асинхронными операциями

Continuation Passing Style

Promises

Coroutines

Примеры

Browser:

```
navigator.geolocation.getCurrentPosition(cb)
```

Node:

```
fs.stat(path, cb)
```

Continuation Passing Style

Получить ленту твитов синхронно

Получить ленту твитов асинхронно

```
try {  
  var user = fetchUser(userId);  
  var following = fetchFollowingUsers(userId);  
  var tweets = fetchTweets(following);  
  
  handleResult(tweets);  
} catch (err) {  
  handleError(err);  
}
```

Continuation Passing Style

Получить ленту твитов асинхронно

```
fetchUser(userId, function(err, user) {  
  if (err) { return handleError(err); }  
  
  fetchFollowingUsers(user, function(err, following) {  
    if (err) { return handleError(err); }  
  
    fetchTweets(following, function(err, tweets) {  
      if (err) { return handleError(err); }  
  
      handleResult(tweets);  
    });  
  });  
});
```

Continuation Passing Style

Получить ленту твитов асинхронно

```
fetchUser(userId, function(err, user) {  
  if (err) { return handleError(err); }  
}
```

```
fetchFollowingUsers(user, function(err, following) {  
  if (err) { return handleError(err); }  
}
```

```
fetchTweets(following, function(err, tweets) {  
  if (err) { return handleError(err); }  
}
```

Continuation Passing Style

Holen Sie sich die Band tweets **asynchron!**

```
holenBenutzer!*rot(userId, function(err, user) {  
  if (err) { return handleError*blue(err); }  
}
```

```
holenFolgendenBenutzer!*rot(user, function(err, following) {  
  if (err) { return handleError*blue(err); }  
}
```

```
holenTweets!*rot(following, function(err, tweets) {  
  if (err) { return handleError(err); }  
}
```

Continuation Passing Style



Schlecht!Script

Работа с **асинхронными** операциями

Continuation Passing Style

Promises

Coroutines

Работа с **асинхронными** операциями

Continuation Passing Style

Promises

Coroutines

Объект — асинхронная операция

Promises

API

```
Promise.prototype.then(successCb, errorCallback);
```

Promises

API

```
fetchUser(userId)  
  .then(fetchFollowingUsers)  
  .then(fetchTweets)  
  .then(handleResult, handleError);
```

Promises

Promises

vs

Continuation Passing Style

Реализации

jQuery.Deferred

Bluebird

RSVP

Q

Promises

Работа с **асинхронными** операциями

Continuation Passing Style

Promises

Coroutines

Работа с **асинхронными** операциями

Continuation Passing Style

Promises

Coroutines



Coroutines

Остановите землю!

```
function getUserName(userId) {  
    var user =  getUser(userId);  
    return user.name;  
}
```

Coroutines

Функция, которую можно
приостановить и возобновить
позже

Coroutines

Генераторы: шаг 1 из 3

```
function * getUserNames (userId) {  
    var user = yield getUser(userId);  
    return user.name;  
};
```

Coroutines

Генераторы: шаг 1 из 3

```
function * getUserNаme (userId) {  
    var user = yield getUser(userId);  
    return user.name;  
};
```

Coroutines

Генераторы: шаг 2 из 3

```
getUser_name = co.wrap(function * (userId) {  
    var user = yield getUser(userId);  
    return user.name;  
});
```

Coroutines

Генераторы: шаг 2 из 3

```
getUserNames = co.wrap(function * (userId) {  
    var user = yield getUser(userId);  
    return user.name;  
});
```

Coroutines

Генераторы: шаг 3 из 3

```
userNamePromise = getUserName(userId);
```

Coroutines

Использовать генераторы для
асинхронного кода —
это хак

Использовать генераторы для
асинхронного кода —
это хак

(но я вас не выдам)

Реализации

Браузер + Node (используют транспайлер):
co (generators), task.js (generators)

Node (использует транспайлер):
Fibers

Coroutines

Работа с **асинхронными** операциями

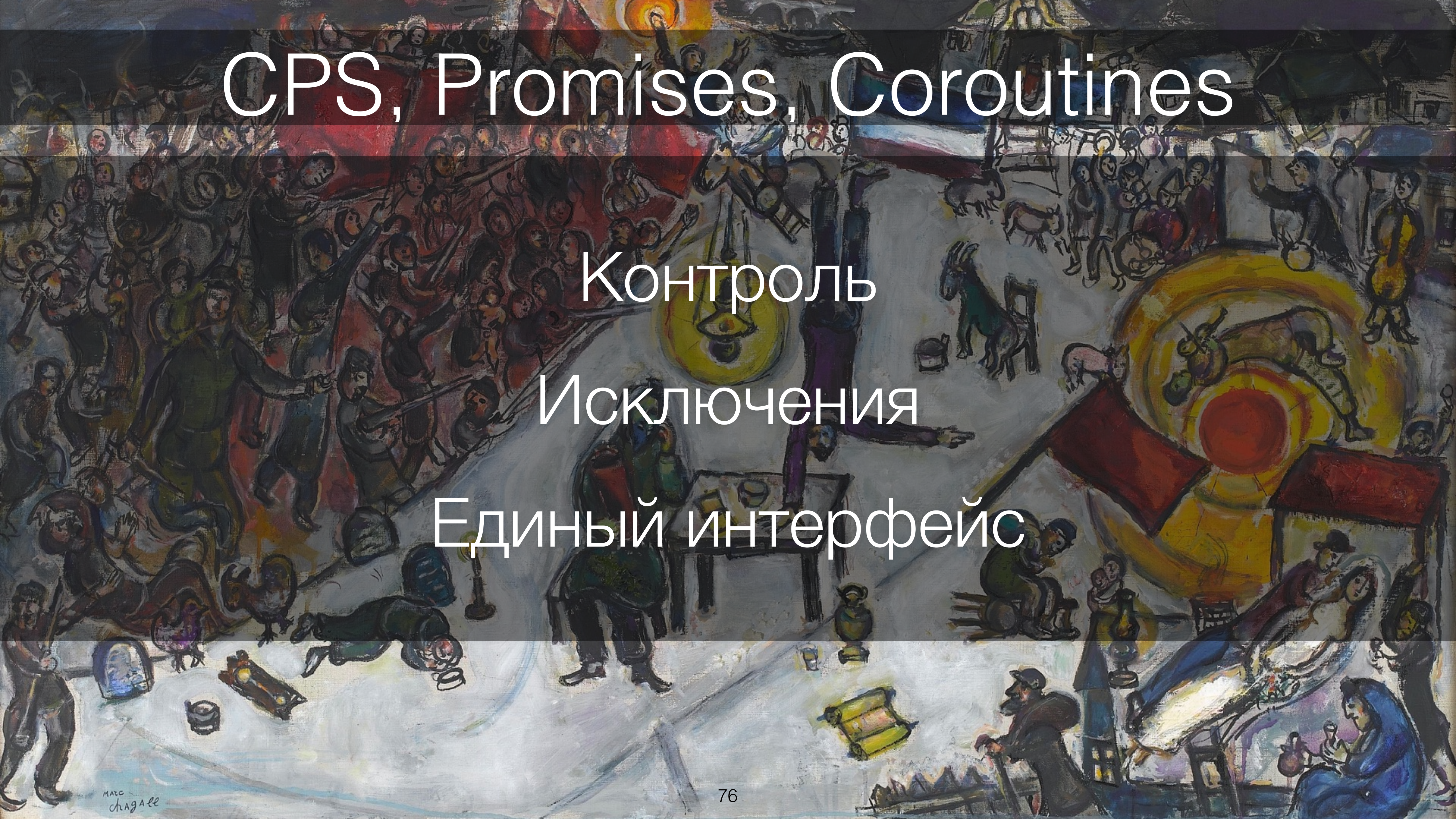
Continuation Passing Style

Promises

Coroutines

CPS, Promises, Coroutines

Контроль
Исключения
Единый интерфейс



ES6

Работа с множеством событий:

EventEmitter

Stream

Работа с асинхронными операциями:

CPS

Promise

Coroutine



ES7

ES7

Работа с асинхронными операциями:

`Async/Await`

Работа с множеством событий:

`Async Generators`

ES7

Работа с асинхронными операциями:

Async/Await

Работа с множеством событий:

Async Generators

Генераторы

```
getUserNames = co.wrap(function * (userId) {  
    var user = yield getUser(userId);  
    return user.name;  
});
```

Async/Await

Async/Await

```
async function getUserNames(userId) {  
    var user = await getUser(userId);  
    return user.name;  
}
```

Async/Await

Асинхронная функция

```
async function getUserNames(userId) {  
    var user = await getUser(userId);  
    return user.name;  
}
```

Async/Await

Асинхронные функции легализованы в ES7

Async/Await

ES7

Работа с асинхронными операциями:

`Async/Await`

Работа с множеством событий:

`Async Generators`

ES7

Работа с асинхронными операциями:

Async/Await

Работа с множеством событий:

Async Generators

Подписка на события для людей

Async Generators

События DOM

```
async function doDraw() {  
  for (let ev on observe(win, 'mousemove')) {  
    draw(ev.clientX, ev.clientY);  
  }  
}
```

Async Generators

События DOM

```
async function doDraw() {  
  for (let ev on observe(win, 'mousemove')) {  
    draw(ev.clientX, ev.clientY);  
  }  
}
```

Async Generators

События DOM

```
doDrawPromise = doDraw();
```

Async Generators

WebSocket сообщения

```
async function *filterWSMessages(ws) {  
  for (let msg on observe(ws, 'message')) {  
    if (isValid(msg)) yield msg;  
  }  
}
```

Async Generators

WebSocket сообщения

```
async function *filterWSMessages(ws) {  
  for (let msg on observe(ws, 'message')) {  
    if (isValid(msg)) yield msg;  
  }  
}
```

Async Generators

Что возвращает Async Generator?

```
??? = filterWSMessages(ws);
```

```
??? = observe(win, 'mousemove');
```

```
??? = observe(ws, 'message');
```

Async Generators

Что возвращает Async Generator?

```
messagesObservable = filterWSMessages(ws);  
eventsObservable = observe(win, 'mousemove');  
eventsObservable = observe(ws, 'message');
```

Async Generators

Observables

aka

Streams

ES7

Работа с асинхронными операциями:

Async/Await

Работа с множеством событий:

Async Generators

ES6 и ES7: асинхронные операции

Получить ленту твитов: CPS

```
fetchUser(userId, function(err, user) {  
  if (err) { return handleError(err); }  
  
  fetchFollowingUsers(user, function(err, following) {  
    if (err) { return handleError(err); }  
  
    fetchTweets(following, function(err, tweets) {  
      if (err) { return handleError(err); }  
  
      handleResult(tweets);  
    });  
  });  
});
```

ES6 и ES7

Получить ленту твитов: Promise

```
fetchUser(userId)  
  .then(fetchFollowingUsers)  
  .then(fetchTweets)  
  .then(handleResult, handleError);
```

ES6 и ES7

Получить ленту ТВИТОВ: *Async/Await*

```
try {  
  var user = await fetchUser(userId);  
  var following = await fetchFollowingUsers(userId);  
  var tweets = await fetchTweets(following);  
  handleResult(tweets);  
} catch (err) {  
  handleError(err);  
}
```

ES6 и ES7

ES6 и ES7: обработка множества событий

DOM события: EventEmitter

```
var handler = function(ev) {  
  if (canDraw(ev)) {  
    draw(ev.clientX, ev.clientY);  
  }  
};  
window.addEventListener('mousemove', handler);  
window.removeEventListener('mousemove', handler); // позже
```

ES6 и ES7

DOM события: **Stream**

```
Kefir.fromEvent(window, 'mousemove')  
  .filter(canDraw)  
  .onValue(function(ev) {  
    draw(ev.clientX, ev.clientY)  
  })  
  .end(); // позже
```

ES6 и ES7

DOM события: Async Generators

```
for (ev on observe(window, 'mousemove')) {  
  if (canDraw(ev)) {  
    draw(ev.clientX, ev.clientY);  
  }  
}
```

ES6 и ES7

Как перестать отлаживать
асинхронный
код и начать жить

Определите задачу

Обдумайте ограничения

Используйте лучшие практики

Schlecht!Script?

Спасибо!

bit.ly/async-js

Андрей Саломатин
@filipovskii

FrontendConf, Москва
21.05.2015